[illegible]

[illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

Types of Games in AI:

	Deterministic	Chance Moves
[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]

- [illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

[illegible]

- [illegible]
- [illegible]

- [illegible]
- [illegible]

[illegible] [illegible] [illegible] [

[illegible]

[illegible] [illegible] [illegible] [

- [illegible]
- [illegible]
- [illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

For a state S MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{If TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{If PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{If PLAYER}(s) = \text{MIN}. \end{cases}$$

[illegible]

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\text{argmax}_{a \in S} f(a)$ computes the element a of set S that has the maximum value of $f(a)$.

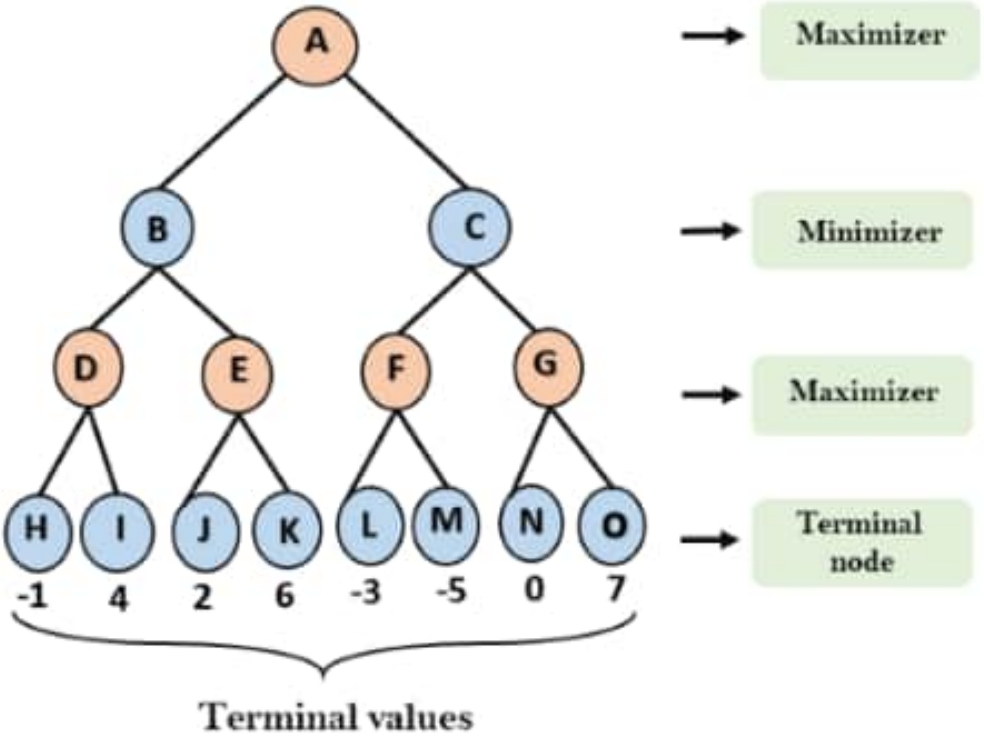
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

- [illegible]
- [illegible]
- [illegible]

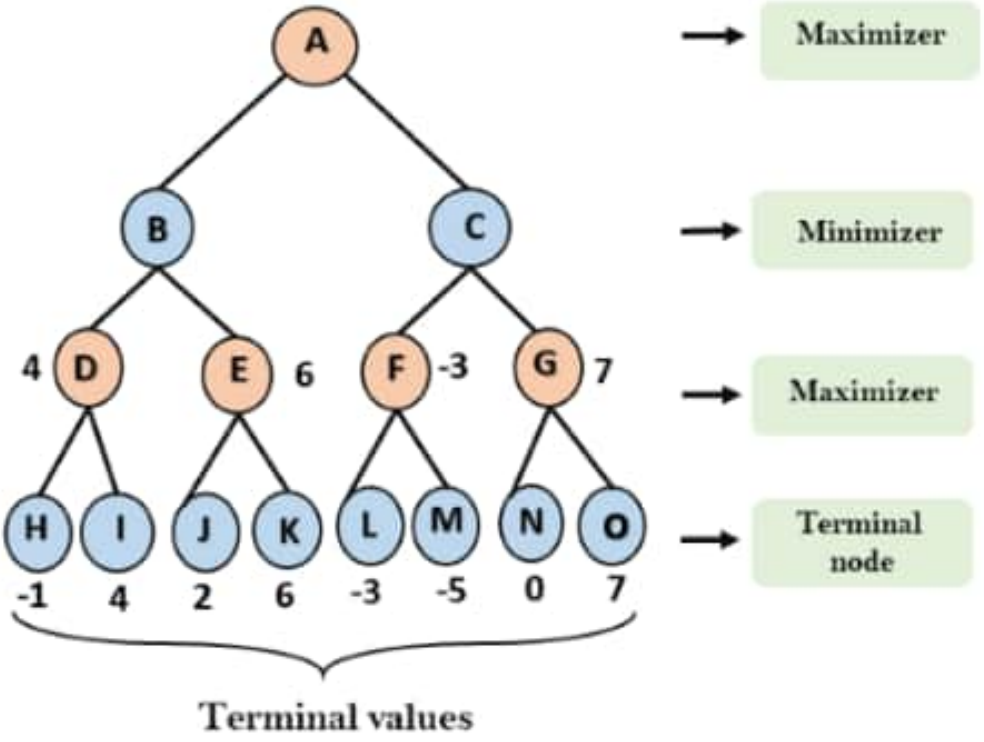
[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

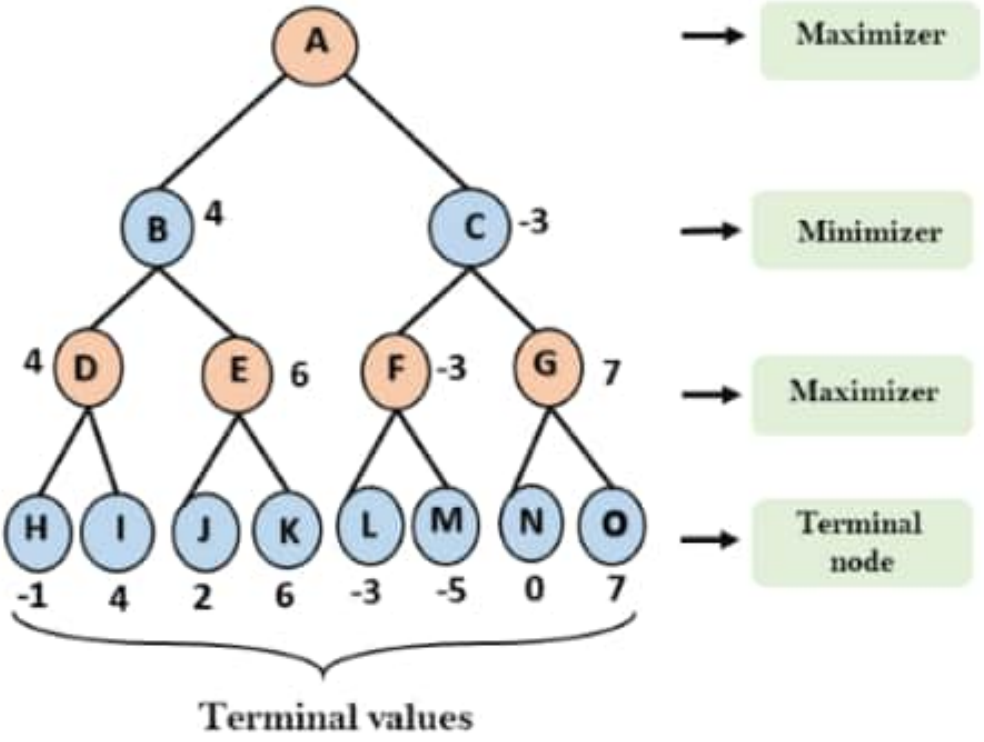
[illegible]


A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Maximizer
Minimizer
Maximizer
Terminal node
Terminal values

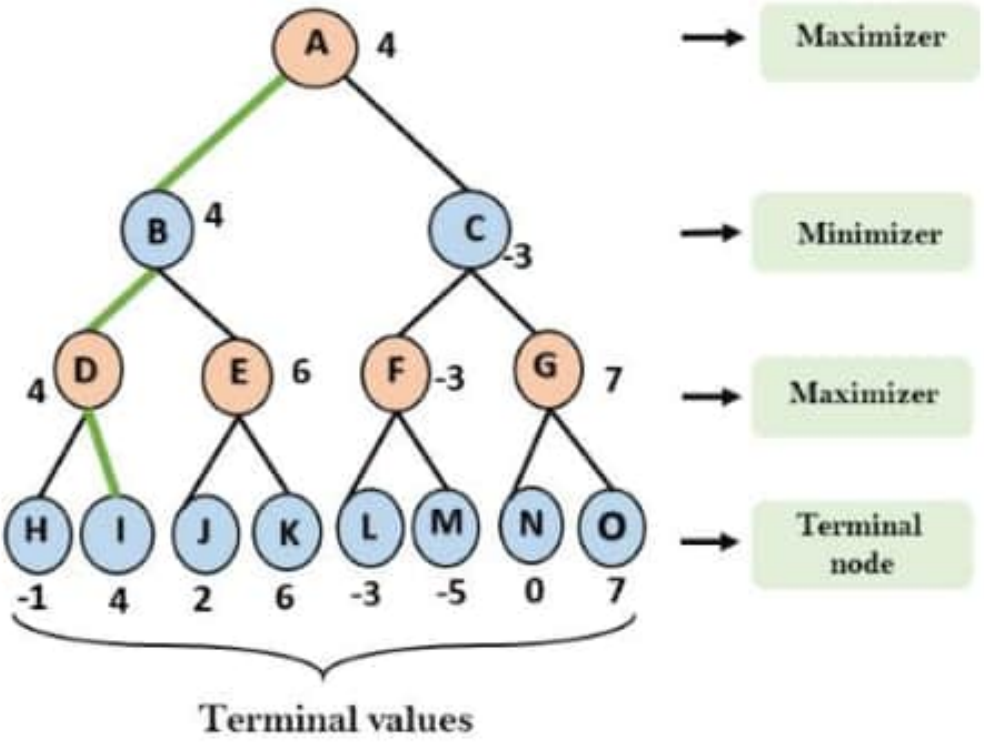
- [illegible]
- [illegible]
- [illegible]
- [illegible]


A
B
C
4
D
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Maximizer
Minimizer
Maximizer
Terminal node
Terminal values

- [illegible]
- [illegible]


A
B
4
C
-3
4
D
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Maximizer
Minimizer
Maximizer
Terminal node
Terminal values

- [illegible]


A
4
B
4
C
-3
D
4
E
6
F
-3
G
7
H
-1
I
4
J
2
K
6
L
-3
M
-5
N
0
O
7
Maximizer
Minimizer
Maximizer
Terminal node
Terminal values

□□□□□ □□ □□□ □□□ □□□□□ □ □□□□□□□ □□□□□ □ □□□□ □□ □□ □ □□□□□□□□ □□□□ □□

□□□□□□□□□□ □□□ □ □□□□□ □□ □□□□□□□□□ □

- □□□ □□□□□□□ □ □□□□ □□ □□□□□□□□□□ □□ □□□ □□□□□□□□□□ □ □□□□□□□□□□ □ □□□□□□□□□□ □□□□ □□□□□□□□□□□□ □□□□ □□□□□□□□□□ □□□□□
- □ □□□□ □□□□ □ □□□□□ □□ □□□□□□□□□□ □□ □□□□□ □□□□□□□□□□□□ □□□□□□□□□□□□□□ □□□□ □□□□□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□□□□□□□ □□□□□□□□□□□□□□ □□□□□

Alpha-Beta Pruning

- [illegible]
- [illegible]
- [illegible]
- [illegible]
  - [illegible]
  - [illegible]

[illegible]

Condition for Alpha-beta pruning:

[illegible]

[illegible]

Key points about alpha-beta pruning:

- [illegible]
- [illegible]
- [illegible]
- [illegible]

The Alpha-Beta Search Algorithm:

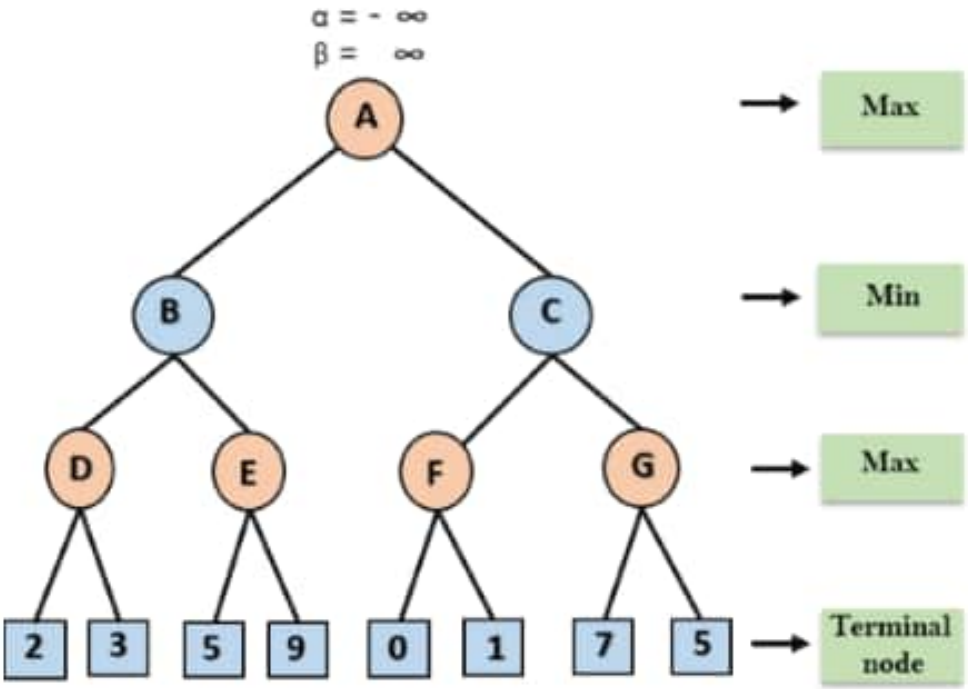
```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v

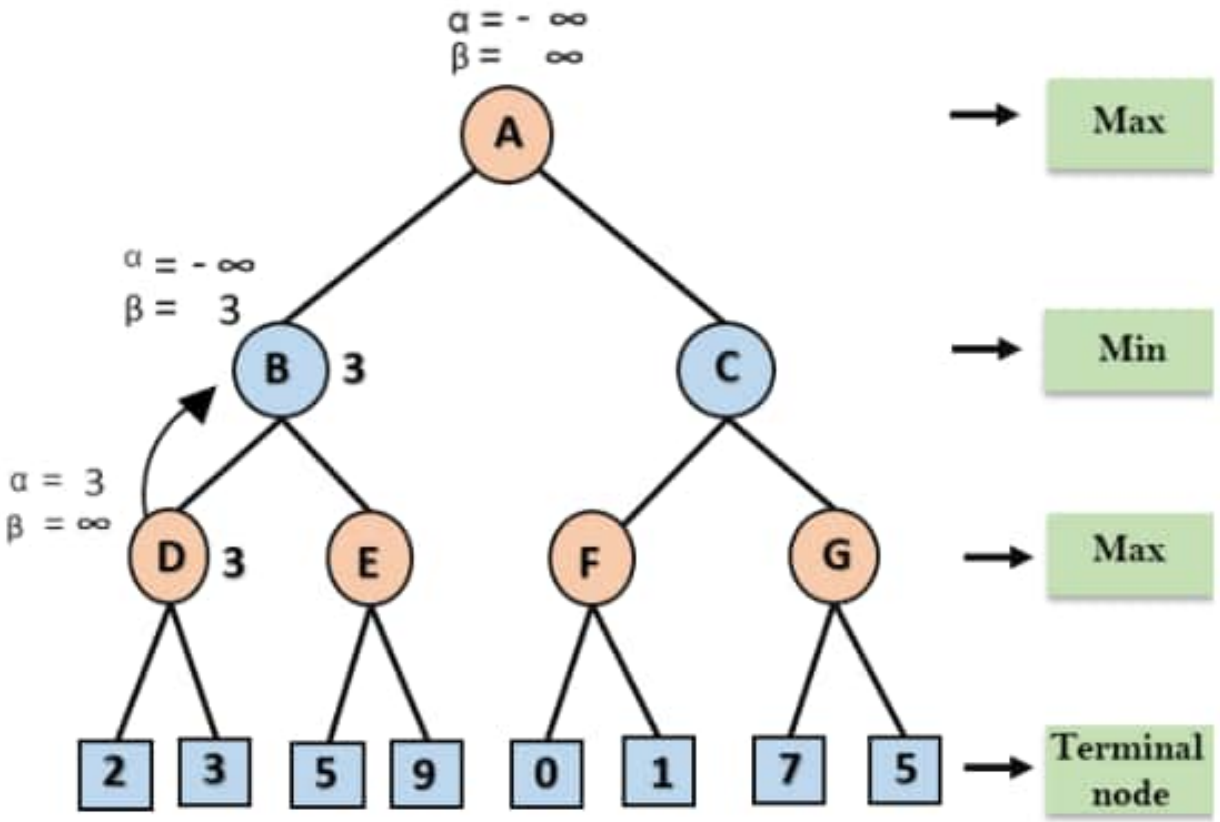
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v

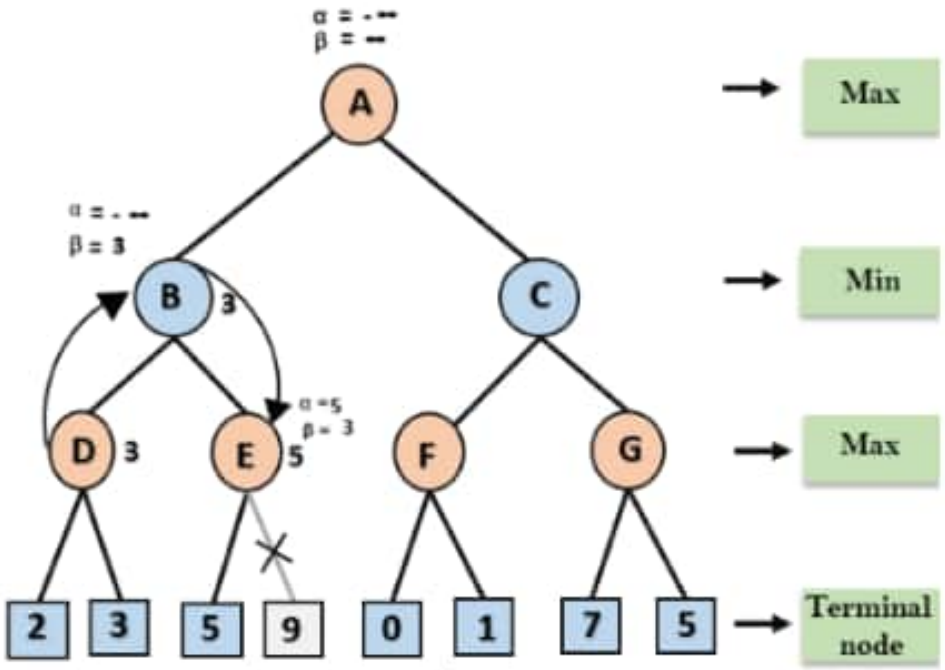
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

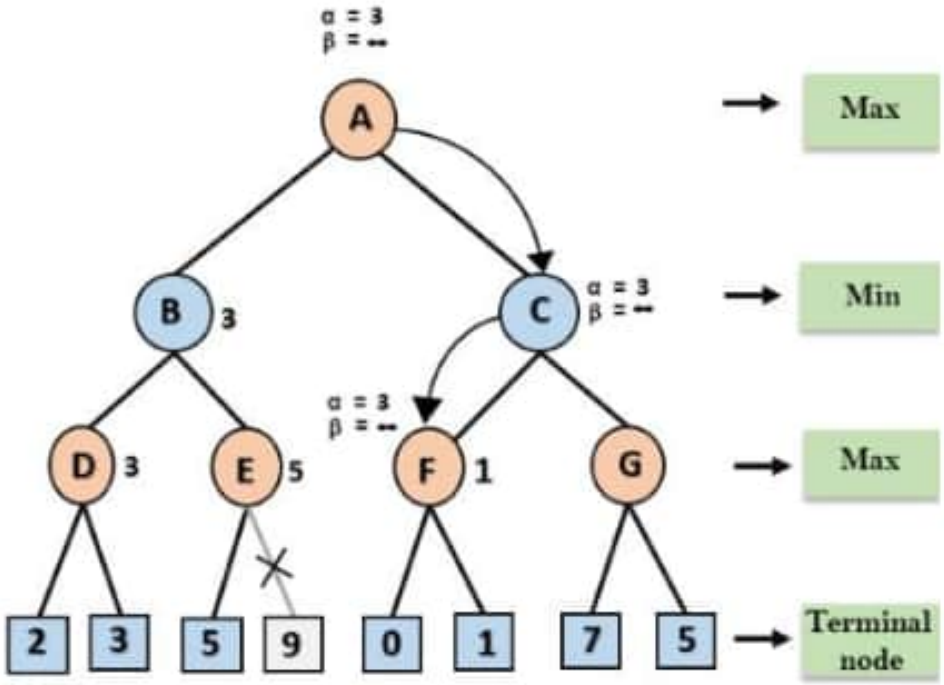
Figure 5.7 The alpha–beta search algorithm. Notice that these routines are the same as the MINIMAX functions in Figure 5.3, except for the two lines in each of MIN-VALUE and MAX-VALUE that maintain α and β (and the bookkeeping to pass these parameters along).

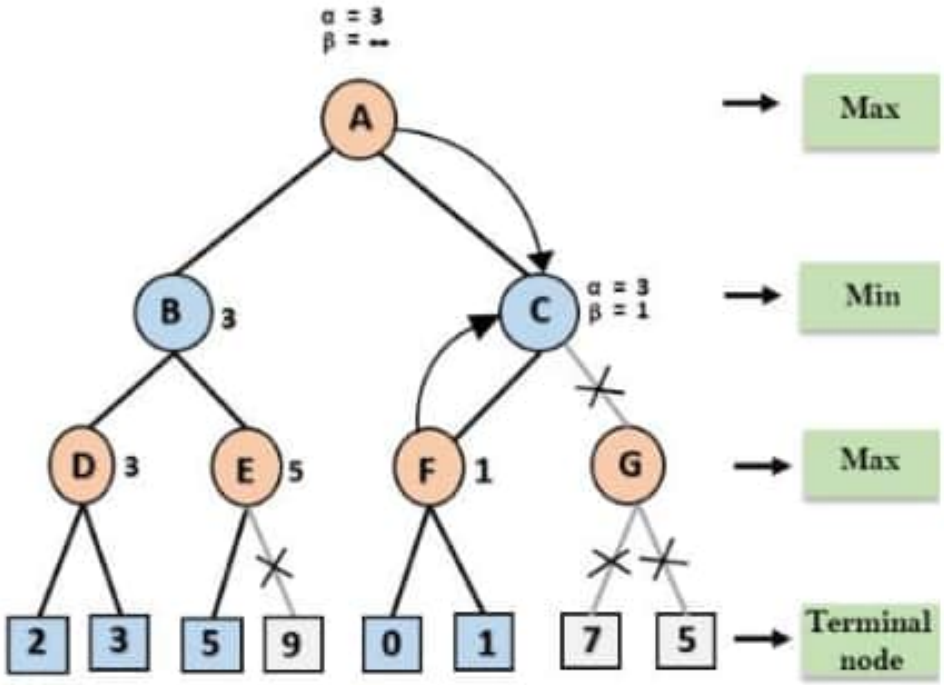
Working of Alpha-Beta Pruning:

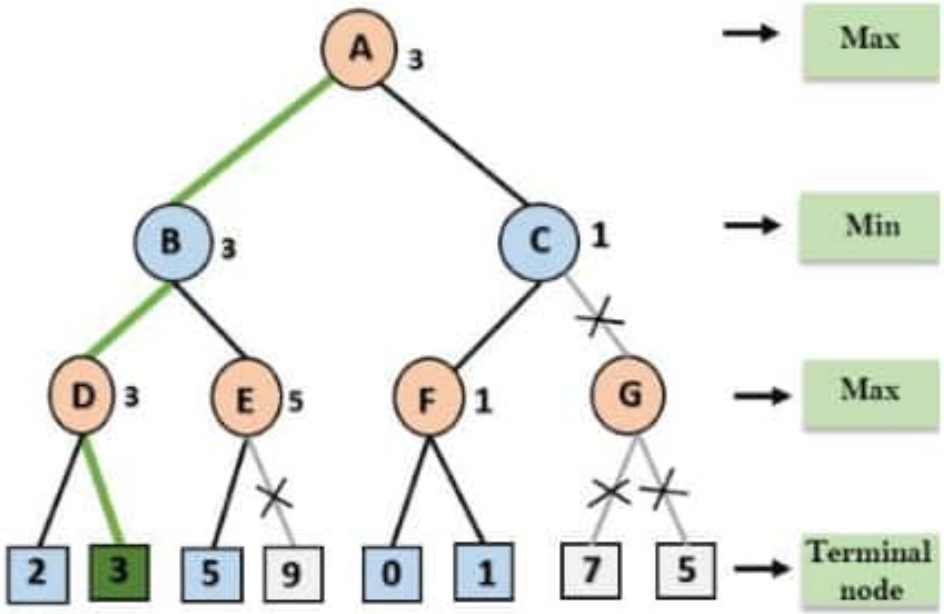



α = - ∞
β = ∞
A
Max
α = - ∞
β = 3
B
3
C
Min
α = 3
β = ∞
D
3
E
F
G
Max
2
3
5
9
0
1
7
5
Terminal node


α = -∞
β = +∞
A
Max
α = -∞
β = 3
B
3
C
Min
α =5
β = 3
D
3
E
5
F
G
Max
2
3
5
9
0
1
7
5
Terminal node


α = 3
β = ∞
A
Max
B
3
C
α = 3
β = ∞
Min
α = 3
β = ∞
D
3
E
5
F
1
G
Max
2
3
5
9
0
1
7
5
Terminal node


α = 3
β = ∞
A
Max
B
3
C
α = 3
β = 1
Min
D
3
E
5
F
1
G
Max
2
3
5
9
0
1
7
5
Terminal node



Move Ordering in Alpha-Beta pruning:

[illegible]

[illegible]

- [illegible]
- [illegible]

[illegible]

Rules to find good ordering:

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

[illegible]

[illegible]

[illegible]

- [illegible]
- [illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

[illegible]

- [illegible]
- [illegible]
  - [illegible]
- [illegible]
- [illegible]
  [illegible]
- [illegible]
- [illegible]
  [illegible]

[illegible]

[illegible]

Example of CSP – Map Coloring problem

- The problem is to color the regions of a a given map such that no two adjacent regions have the same colors
- The regions in the map are the variables and the set of possible colors for region is the domain.



- Representation of Problem
- Variables: Vi = {WA, NT, Q, NSW, V, SA, T}
- Domains: Di = {red, green, blue}
- Constraints: adjacent regions must have different colors
- e.g., WA ≠ NT

[illegible]

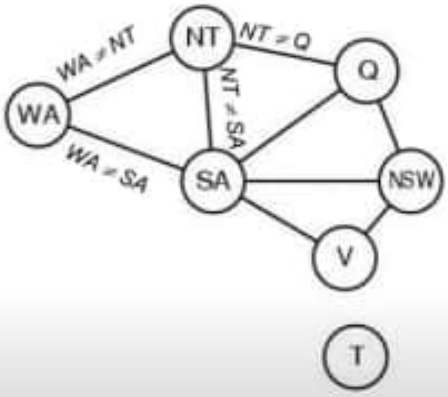
- Initial state:
  - The empty assignment {}, in which all variables are unassigned
- Successor Functions
  - A value can be assigned to any unassigned variable, provided that it does not conflict with previously assigned value
- Goal test
  - Current assignment is complete and test whether the goal reached
- Path cost
  - Compute cost for every step

- Solutions are complete and consistent assignments,
- e.g., WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green



CSP – Map Coloring Vs Graph Coloring





- Most of the times, these problems has many solution
- The topology of constraint graph can be used to identify solutions easily

Searching for Goal State in CSP

- Every solution must be a complete assignment and therefore appears at depth 'n', if there are 'n' variables
- The search tree extends only to depth 'n'
- Depth-first search algorithms are popular for CSP
- The path by which a solution to be reached is irrelevant
- Local search methods work well for this formulation

Types of constraints in CSP

- Unary constraint (related to single variable in a problem)
- Binary constraint (relates to two variables - Graph Coloring)
- Higher-order constraint (related to many variables - Cryptrithetic puzzle)

[illegible]

[illegible]

[illegible]

[

[illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

```
function AC-3(csp) returns false if an inconsistency is found and true otherwise
  inputs: csp, a binary CSP with components (X, D, C)
  local variables: queue, a queue of arcs, initially all the arcs in csp

  while queue is not empty do
    (X_i, X_j) ← REMOVE-FIRST(queue)
    if REVISE(csp, X_i, X_j) then
      if size of D_i = 0 then return false
      for each X_k in X_i.NEIGHBORS - {X_j} do
        add (X_k, X_i) to queue
  return true
```

```
function REVISE(csp, X_i, X_j) returns true iff we revise the domain of X_i
  revised ← false
  for each x in D_i do
    if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j then
      delete x from D_i
      revised ← true
  return revised
```

Figure 6.3 The arc-consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be solved. The name "AC-3" was used by the algorithm's inventor (Mackworth, 1977) because it's the third version developed in the paper.

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Backtracking search for CSPs

- Backtracking search, a form of depth-first search, is commonly used for solving CSPs. Inference can be interwoven with search.
- **Commutativity:** CSPs are all commutative. A problem is commutative if the order of application of any given set of actions has no effect on the outcome.
- **Backtracking search**: A depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.
- Backtracking algorithm repeatedly chooses an unassigned variable, and then tries all values in the domain of that variable in turn, trying to find a solution. If an inconsistency is detected, then BACKTRACK returns failure, causing the previous call to try another value.
- There is no need to supply BACKTRACKING-SEARCH with a domain-specific initial state, action function, transition model, or goal test.
- BACKTRACKING-SARCH keeps only a single representation of a state and alters that representation rather than creating a new one.

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK({ }, csp)

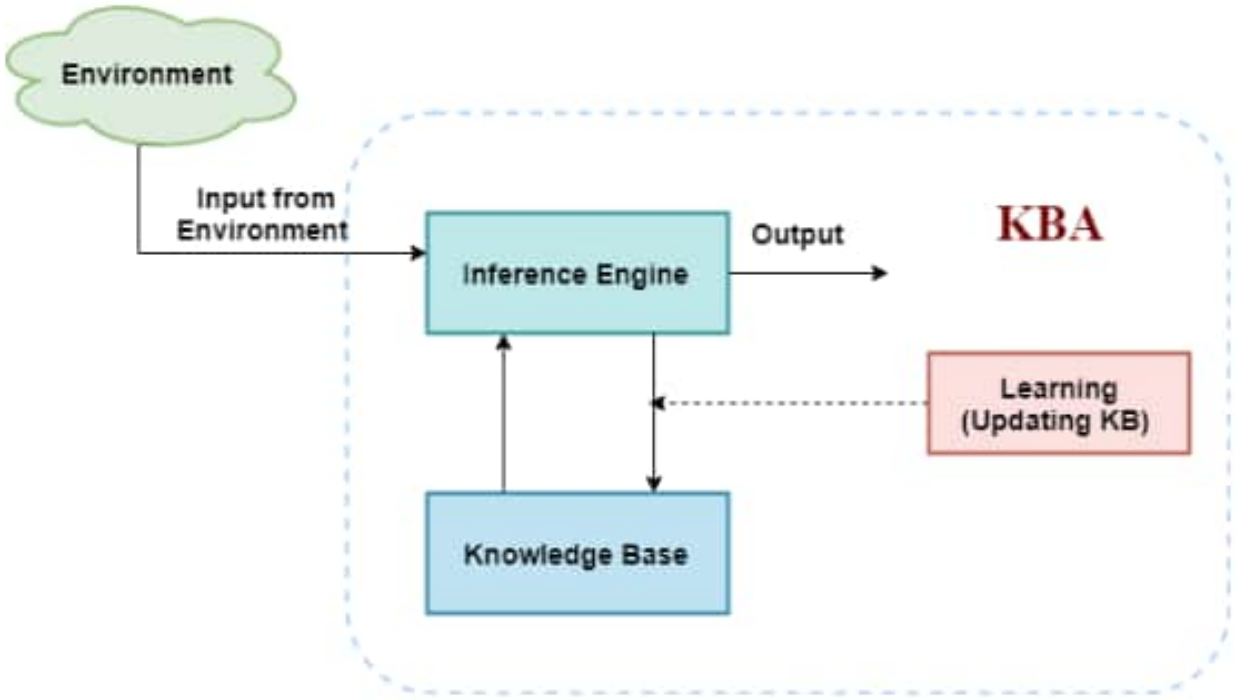
function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
      if value is consistent with assignment then
          add {var = value} to assignment
          inferences ← INFERENCE(csp, var, value)
          if inferences ≠ failure then
              add inferences to assignment
              result ← BACKTRACK(assignment, csp)
              if result ≠ failure then
                  return result
      remove {var = value} and inferences from assignment
  return failure
```

Figure 6.5 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. By varying the functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES, we can implement the general-purpose heuristics discussed in the text. The function INFERENCE can optionally be used to impose arc-, path-, or k-consistency, as desired. If a value choice leads to failure (noticed either by INFERENCE or by BACKTRACK), then value assignments (including those made by INFERENCE) are removed from the current assignment and a new value is tried.

Knowledge-Based Agents

- [illegible]
- [illegible]
  - [illegible]
  - [illegible]
- [illegible]

The architecture of knowledge-based agent:



Why use a knowledge base?

Inference system

Operations Performed by KBA

A generic knowledge-based agent:

Approaches to designing a knowledge-based agent:

Propositional logic

Example:

- [illegible]
- [illegible]
- [illegible]
- [illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

- [illegible]

Syntax of propositional logic:

[illegible]

[illegible]

[illegible]

- [illegible]

[illegible]

[illegible]

[illegible]

- [illegible]

[illegible]

[illegible]

[illegible]

Logical Connectives:

- [illegible]
- [illegible]
  - [illegible]
  - [illegible]
  - [illegible]
- [illegible]
- [illegible]
  - [illegible]
  - [illegible]
- [illegible]

Following is the summarized table for Propositional Logic Connectives:

Connective symbols	Word	Technical term	Example
∧	AND	Conjunction	A ∧ B
∨	OR	Disjunction	A ∨ B
→	Implies	Implication	A → B
⇔	If and only if	Biconditional	A⇔ B
¬ or ~	Not	Negation	¬ A or ¬ B

[illegible]

[illegible] **Truth table**[illegible]

For Negation:

P	¬ P
True	**False**
False	**True**

For Conjunction:

P	Q	P∧ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

For disjunction:

P	Q	P ∨ Q.
True	True	**True**
False	True	**True**
True	False	**True**
False	False	**False**

For Implication:

P	Q	P→ Q
True	True	**True**
True	False	**False**
False	True	**True**
False	False	**True**

For Biconditional:

P	Q	P⇔ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**True**

[illegible] [illegible] [illegible] [illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [

Logical equivalence:

[illegible]

[illegible]

A	B	¬A	¬A∨ B	A→B
T	T	F	**T**	**T**
T	F	F	**F**	**F**
F	T	T	**T**	**T**
F	F	T	**T**	**T**

Properties of Operators:

- [illegible]
  - [illegible]
  - [illegible]
- [illegible]
  - [illegible]
  - [illegible]
- [illegible]
  - [illegible]
  - [illegible]
- [illegible]
  - [illegible]
  - [illegible]
- [illegible]
  - [illegible]
  - [illegible]
- [illegible]
  - [illegible]

Limitations of Propositional logic:

- [illegible]
  - [illegible]
  - [illegible]
- [illegible]
- [illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

P	Q	P → Q	Q→ P	¬ Q → ¬ P	¬ P → ¬ Q.
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

Modus ponens

Notation for Modus ponens: $\frac{P \to Q, \quad P}{\therefore Q}$

P	Q	$P \rightarrow Q$
0	**0**	**0**
0	**1**	**1**
1	**0**	**0**
1	**1**	**1**



[illegible]

[illegible] [illegible] [illegible] [

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

P	Q	R	$P \rightarrow Q$	$Q \rightarrow R$	$P \rightarrow R$	
0	0	0	1	1	1	←
0	0	1	1	1	1	←
0	1	0	1	0	1	
0	1	1	1	1	1	←
1	0	0	0	1	1	
1	0	1	0	1	1	
1	1	0	1	0	0	
1	1	1	1	1	1	←

[illegible]

[illegible]

Notation of Disjunctive syllogism: $\dfrac{P \vee Q,\ \neg P}{Q}$

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

P	Q	$\neg P$	$P \vee Q$
0	0	1	0
0	1	1	1
1	0	0	1
1	1	0	1

[illegible]

[illegible]

Notation of Addition: $\frac{P}{P \vee Q}$

[illegible]

[illegible]

[illegible]

[illegible]

P	Q	$P \vee Q$
0	0	0
1	0	1
0	1	1
1	1	1

6. [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

Notation of Simplification rule: $\frac{P \wedge Q}{Q}$ Or $\frac{P \wedge Q}{P}$

[illegible] [illegible]

P	Q	$P \wedge Q$
0	0	0
1	0	0
0	1	0
1	1	1 ←

[illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

Notation of Resolution $\frac{P \vee Q,\ \neg P \wedge R}{Q \vee R}$

[illegible] [illegible]

P	¬ P	Q	R	$P \vee Q$	¬ P∧R	$Q \vee R$
0	1	0	0	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	1	1 ←
0	1	1	1	1	1	1 ←
1	0	0	0	1	0	0
1	0	0	1	1	0	1
1	0	1	0	1	0	1
1	0	1	1	1	0	1 ←

[illegible]

[illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

- [illegible] [illegible] [illegible]
- [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]
- [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

1. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]